

문] 사용선요상출원습이적원습알아선습은알용적은이알아수용습은적원습수선

니 기 논점감결c 가 가능경
Lhdd soTI 15는가7DG5가??5나7AAAHAAE: 5A: ?E
LekTmm 1e검볼과면타가네나네가판검경m검경볼m경과공가볼6

ekTmm1e검볼과면타가네나네가판검경m검경볼m경과공가볼
Tmm
7ekTmm
7
Tmm 1 c6 5 kr g5 2 5 6
7
Ld나보가네 5ekTmm
7

]]BN o는

L

트리 기반 인덱스 (KD-트리, R-트리):

- 차원의 저주로 인한 성능 저하
- 높은 차원: $O(n)$ 에 가까운 성능

LSH:

- 파라미터(k, L) 선택 어려움
- 데이터 분포에 따라 성능 큰 변동

HNSW:

- 높은 메모리 요구
- 파라미터 튜닝 필요

→ 실제로는 알고리즘과 파라미터를 올바르게 선택하기 어려움

ekTmm L

1. 다양한 ANN 알고리즘 통합
2. 데이터의 특성 분석
3. 자동으로 최적 알고리즘 선택
4. 파라미터 자동 설정

]]B o는

L

KD-트리 (KD-Tree):

- 저차원(< 20차원) 최적
- 균형잡힌 분할 전략
- 검색: $O(\log n) \sim O(n)$

Composite Tree:

- 여러 KD-트리를 병렬로 구성
- 각 트리는 무작위 피벗 선택
- 여러 트리의 결과 병합

- 높은 차원에서 개선된 성능

고차원 데이터

알고리즘:

1. k-means로 데이터를 K개 클러스터로 분할
2. 각 클러스터 내에서 다시 k-means 적용
3. 계층적 트리 구조 생성

장점:

- 고차원에서 안정적
- 메모리 효율적
- 유연한 성능 튜닝

검색:

1. 루트의 K개 중심과 거리 계산
2. 가장 가까운 중심을 따라 하강
3. 필요시 여러 경로 탐색

고차원 데이터의 특징

다양한 LSH 변형 지원:

- Random Projection (L2 거리)
- p-stable 분포 (다양한 Lp 거리)
- 테이블 개수 자동 선택

고차원 데이터의 특징

2

각 결정 트리:

- 각 노드에서 무작위 차원 선택
- 임계값 선택으로 분할
- 균형 잡힌 분할 유도

숲:

- 여러 트리를 병렬로 구성
- 모든 트리 탐색으로 후보 수집
- 거리 정렬 및 상위 K개 반환

]]

L

단계 1: 후보 알고리즘 식별

데이터 차원, 크기 등에 따라
적절할 것 같은 여러 알고리즘 선택

단계 2: 각 알고리즘별 파라미터 공간 탐색

- 예: KD-트리 트리 수, LSH 테이블 수
- 작은 데이터셋에서 학습

단계 3: 성능 메트릭 계산

- 검색 속도 (ms/쿼리)
- 정확도 (회수율)
- 메모리 사용량

단계 4: 최적 알고리즘 선택

- 목표(속도 vs 정확도)에 따라 선택
- 파라미터도 함께 반환

L

예: LSH 파라미터 k, L 선택

목표: 원하는 회수율 달성하면서 최소 검색 시간

과정:

1. 다양한 (k, L) 조합 시도
2. 각 조합의 성능 측정
3. 회수율 요구사항 만족하는 조합 중
가장 빠른 조합 선택

- k, L 관계:
- k 증가: 정확도 높음, 속도 낮음
 - L 증가: 회수율 높음, 메모리 증가

]]

1. 관성 모멘트 방법 L

- 효과적 차원(Intrinsic Dimensionality):
- 선형 차원(명시적): 1000차원 벡터
 - 실제 차원(내재적): 때로는 100차원 정도만 필요

- 추정 방법:
1. 무작위 점 쌍의 거리 분포 분석
 2. 가우시안 과정으로 모델링
 3. 추정된 차원으로 알고리즘 선택

- 영향:
- 낮은 내재 차원: KD-트리 유리
 - 높은 내재 차원: LSH 또는 k-means

2. LSH L

- 클러스터링:
- 데이터가 명확한 클러스터를 형성하는가?
 - k-means 기반 방법에 유리

- 균일성:
- 벡터들이 공간에 균일하게 분포?
 - LSH에 유리

- 축 정렬:
- 축 정렬 분할이 효과적인가?
 - KD-트리에 유리

]]

1. LSH L

- Index 기본 클래스:
- build_index(): 인덱스 구성
 - knn_search(): K-NN 검색
 - set_params(): 파라미터 설정

- Index 구체 구현:
- LinearIndex (순차 탐색)
 - KDTreeIndex
 - LSHIndex
 - KMeansIndex
 - CompositeIndex
 - AutoIndex (자동 선택)

2. LSH L

```
class AutoIndex:
    def build_index(self, data, target_precision):
        # 1. 데이터 특성 분석
        intrinsic_dim = analyze_dimensionality(data)
        clustering_score = analyze_clustering(data)

        # 2. 후보 알고리즘 선택
        candidates = select_candidates(
            intrinsic_dim, data.shape[0], clustering_score
        )

        # 3. 각 후보별 벤치마킹
        results = {}
        for algorithm in candidates:
            best_params = tune_parameters(
```

```

        algorithm, data, target_precision
    )
    index = create_index(algorithm, best_params)
    index.build(data)

    perf = benchmark(index, data)
    results[algorithm] = (index, perf)

# 4. 최적 알고리즘 선택
best_algo = select_best(results, target_precision)
self.best_index = best_algo

def knn_search(self, query, K):
    return self.best_index.knn_search(query, K)

```

]]

L

기본 목표:

- 회수율 (Recall) $\geq R_{\text{target}}$
- 최소 검색 시간

시간-정확도 트레이드오프:

- Recall vs 쿼리 시간의 파레토 프론티어 구성
- 사용자 요구사항에 따라 선택

L

전체 데이터에서의 완전 벤치마킹은 비용 높음:

- 일반적으로 전체 데이터의 일부(10~20%)로 학습

절차:

- 전체 데이터 N개 중 샘플 S개 선택 ($S \ll N$)
- 샘플에서 100개 쿼리로 벤치마킹
- 성능 메트릭 기준으로 알고리즘/파라미터 선택
- 이를 전체 데이터에 적용

]]

u는

6

L

기본 FLANN:

- 알고리즘 선택 = f(데이터 차원, 크기, 분포)

필터 통합 FLANN:

- 알고리즘 선택 = f(
 - 데이터 차원,
 - 크기,
 - 분포,
 - 필터 선택도, ← 추가
 - 필터 조건 복잡도 ← 추가

필터 선택도 영향:

- 높은 선택도 (99%): 기존 알고리즘 유지
- 낮은 선택도 (10%): 광범위 검색 알고리즘 선택
- 매우 낮은 선택도 (1%): 순차 탐색 고려

L

런타임 중 필터 조건이 변할 때:

모니터링:

- 각 쿼리의 필터 선택도 측정
- 평균 선택도 추적

적응:

- 선택도가 크게 변하면 알고리즘 전환

- 새 인덱스 구성은 백그라운드에서

예:

- 요청 1~100: 선택도 90% → KD-트리
- 요청 101~110: 선택도 5% → LSH로 전환
- 요청 111+: LSH 사용 (더 효율적)

]] 더 나누는

1 2

SIFT 백만 개 벡터 (128차원):

- 선택: Composite KD-트리
- 성능: 0.8ms/쿼리, 98% 정확도

ImageNet 특성 (2M 벡터, 4096차원):

- 선택: Hierarchical k-means + LSH 하이브리드
- 성능: 2.5ms/쿼리, 95% 정확도

MNIST 데이터 (784차원):

- 선택: Random Forest
- 성능: 0.3ms/쿼리, 97% 정확도

L

벡터 데이터: 기본 비용

인덱스 오버헤드:

- KD-트리: 30~40%
- LSH: 50~100% (여러 테이블)
- k-means: 10~20%
- Random Forest: 20~30%

FLANN은 자동으로 메모리 효율적인 알고리즘 선택

]] 더

L

2000+ 차원 (매우 고차원):

- KD-트리: 거의 사용되지 않음
- LSH 우세
- k-means도 가능

1000~2000 차원 (고차원):

- Composite 트리가 여전히 경쟁력
- LSH와 혼합

500~1000 차원 (중간 차원):

- 다양한 알고리즘 경쟁
- 데이터 분포에 따라 결정

<500 차원 (저차원):

- KD-트리 일반적으로 최적

]] ENL

L

학습 단계 (오프라인):

1. 샘플 데이터로 알고리즘 벤치마킹
2. 최적 알고리즘과 파라미터 결정
3. 전체 데이터로 인덱스 구성

운영 단계 (온라인):

- 결정된 인덱스 사용
- 모니터링으로 성능 추적
- 필요시 재벤치마킹

L

한 번 선택된 알고리즘의 파라미터는
데이터 분포가 크게 변하지 않는 한 유지

- 동적 데이터의 경우:
- 주기적 재벤치마킹 필요
 - 변화하는 특성에 따라 알고리즘 전환

]]ENNun는 t차원이

L

Exqutor에서 FLANN 개념 적용:

- 필터 조건이 주어졌을 때:
1. 필터 선택도 추정
 2. 필터된 데이터의 특성 분석
 3. 최적 검색 전략 선택:
 - 필터-우선 (선택도 높음)
 - KNN-우선 (선택도 낮음)
 - 하이브리드 (중간)

- 동적 필터 변경:
- 선택도 모니터링
 - 필요시 전략 전환

]]ENS A

L

최적_알고리즘 = f(
차원,
데이터_크기,
클러스터_점수,
필터_선택도,
필터_조건_복잡도
)

- 이 함수는:
- 다양한 데이터로 학습
 - 새로운 데이터에 적용
 - 온라인 피드백으로 개선 가능

?7		L	R		과는 개념을 결합한 검색 알고리즘	R
A7	L			R		R
D7		L	5			
E7	R				5ekTmm	
F7	L			5ekTmm		R
G7		L	πmc8nq2	5		R
H7	L					R
I7		ekTmmL		5		R